

EINSTIEG IN GENAI

MODELLE, ANWENDUNGEN UND OPTIMIERUNG VON LLMS

HALLO, ICH BIN DER BASTI 🖐️

- Sebastian Springer
- Web-Entwicklung und Dinge mit KI
- München
- Coding mit KI
- Repo

AGENDA

- GenAI Grundlagen - Modelle, Tokens, Kontext, etc.
- Was sind LLMs
- LLMs lokal ausführen
- Arbeiten mit LLMs
- Modelloptimierung

WAS IST GENAI?

WAS IST GENAI?

- Text: ChatGPT
- Bilder: DALL-E
- Videos: Sora
- 3D Modelle: MasterpieceX

KLASSISCHE KI VS. GENAI

- Klassische KI: Eingaben in Kategorien einordnen oder Vorhersagen treffen
 - Bildklassifizierung
 - Spracherkennung
- GenAI: Erzeugen von Inhalten mit Wahrscheinlichkeiten und Mustern

EINSATZGEBIETE

WAS KANN ICH MIT GENAI MACHEN?

- Arbeiten mit Texten
- Arbeiten mit Bildern
- Audio- und Musikgenerierung
- Videogenerierung und Animation
- Datenaugmentation und Simulation
- Personalisierung und Empfehlungssysteme
- Kreativitätsunterstützung

ARBEITEN MIT TEXTEN

- Content Creation: Schreiben von Artikeln, Blogposts oder Social-Media-Posts
 - ChatGPT, Claude, Gemini, DeepSeek, Grok
- Chatbots und virtuelle Assistenten: Verarbeiten natürliche Sprache und generieren Antworten
- Codegenerierung: Erstellen von Code-Schnipseln oder ganzen Programmen
 - CoPilot, Cursor, Continue

ARBEITEN MIT BILDERN

- Bildgenerierung: von einfachen Icons über abstrakte Hintergrundbilder bis fotorealistische Bilder
 - DALL-E, Midjourney, Artbreeder, RunwayML, Stable Diffusion
- Bildanalyse: Beschreibung von Bildern, Erkennung von Objekten oder Personen

AUDIO- UND MUSIKGENERIERUNG

- Musikgenerierung: Komponieren von Melodien, Harmonien oder ganzen Songs
- Sprachsynthese: Erzeugen von Sprachausgaben oder Soundeffekten

VIDEOGENERIERUNG UND ANIMATION

- Videogenerierung: Erstellen von Videos, Trailern oder Werbespots
- Animation: Erzeugen von Animationen, GIFs oder Effekten
- Spezialeffekte: Erzeugen von visuellen Effekten oder Animationen

DATENAUGMENTATION UND SIMULATION

- Datenaugmentation: Erzeugen von Trainingsdaten oder Testdaten (auch für KI Trainings)
- Simulation: Erzeugen von Simulationen oder Szenarien

PERSONALISIERUNG UND EMPFEHLUNGSSYSTEME

- Personalisierung: Anpassen von Inhalten oder Produkten an individuelle Bedürfnisse und Vorlieben z.B. in Webshops oder Apps
- Empfehlungssysteme: Generieren von Produktempfehlungen oder Vorschlägen

KREATIVITÄTSUNTERSTÜTZUNG

- Ideenfindung: Generieren von Ideen, Konzepten oder Designs
- Prototyping: Erstellen von Prototypen oder Entwürfen
- Konzeptentwicklung: Entwickeln von Konzepten oder Strategien

NACHTEILE UND GEFAHREN

NACHTEILE UND GEFAHREN VON GENAI

- Ethik
- Copyright
- Halluzinationen
- Kosten
- Einfluss auf die Arbeitswelt

ETHIK

- Transparenz und Nachvollziehbarkeit: Oft ist nicht erkennbar wie Ergebnisse zustande kommen.
- Bias und Diskriminierung: KI-Systeme können Vorurteile und Diskriminierung verstärken.
- Missbrauch: KI-Systeme können für Manipulationen oder Betrug eingesetzt werden.
- Datenschutz und Privatsphäre: KI-Systeme können sensible Daten verarbeiten.

COPYRIGHT

- Trainingsdaten: Nutzung urheberrechtlich geschützter Inhalte beim Training - z.B. OpenAI vs. New York Times
- Eigentumsverhältnisse: Eigentum an generierten Inhalten ist unklar
- Kommerzielle Nutzung: Unsicherheiten bei kommerzieller Nutzung von generierten Inhalten
- Haftung und Verantwortlichkeit: Ist bei generierten Inhalten teilweise unklar

HALLUZINATIONEN

- Falsche Fakten: Inhalte klingen plausibel, sind aber falsch oder komplett erfunden
- Vertrauensverlust: Vertrauen in Informationen und Quellen kann sinken
 - Risiko in sensiblen Bereichen: ungenaue Daten können schwerwiegende Folgen haben (Medizin, Recht, Finanzen)
 - Verzerrte Entscheidungsgrundlagen: Falsche Informationen können zu falschen Entscheidungen führen

KOSTEN

- Hoher Rechenbedarf: Training und Inferenz benötigen viel Rechenleistung
- Trainingsdaten: Sammeln und Annotieren von Trainingsdaten ist aufwändig und teuer
- Wartung und Support: Betrieb und Wartung von KI-Systemen erfordert spezielles Know-How
- Kosten der Dienste: Nutzung von GenAI-Diensten kann teuer werden

EINFLUSS AUF DIE ARBEITSWELT

- Automatisierung: Durch Automatisierung können Arbeitsplätze verloren gehen
- Qualifikationen: Regelmäßige Schulungen und Weiterbildungen notwendig, um Schritt zu halten
 - Arbeitsbedingungen: Erhöhte Arbeitsgeschwindigkeit und -druck durch Automatisierung
- Unternehmenskultur: Negative Auswirkungen auf zwischenmenschliche Kommunikation und

WELCHE MODELLE GIBT ES?

WELCHE MODELLE GIBT ES?

- Transformer-basierte Sprachmodelle
- Diffusionsmodelle
- Generative Adversarial Networks (GANs)
- Autoregressive Models
- Recurrent Neural Networks (RNNs)
- Reinforcement Learning
- Variational Autoencoders (VAEs)
- Graph Neural Networks (GNNs)
- Normalizing Flows
- Score-Based Generative Models

WELCHE LLMS GIBT ES?

WELCHE LLMS GIBT ES?

- GPT
- Llama
- Qwen
- BERT
- T5
- Claude
- Mistral
- Grok
- Gemini
- Phi
- Gemma

Weitere Informationen zu LLMs

WIE FUNKTIONIERT EIN LLM?

WIE FUNKTIONIERT EIN LLM?

1. Tokenisierung: Der Text wird in kleinere Einheiten, sogenannte Tokens, zerlegt. Diese können Wörter, Wortteile oder Zeichenfolgen sein.
2. Einbettung (Embedding): Jedes Token wird in einen Vektor im kontinuierlichen Raum umgewandelt, wobei semantisch ähnliche Tokens ähnliche Vektoren erhalten.
3. Vorhersage des nächsten Tokens: Das Modell berechnet die Wahrscheinlichkeit für jedes mögliche nächste Token basierend auf dem bisherigen Kontext.
4. Dekodierung: Anhand der berechneten Wahrscheinlichkeiten wird entschieden, welches

WAS SIND PARAMETER

WAS SIND PARAMETER

In einem Large Language Model (LLM) sind Parameter die anpassbaren Werte, die während des Trainings optimiert werden. Sie bestimmen, wie das Modell Sprache versteht und generiert. Die Anzahl der Parameter beeinflusst die Komplexität und Leistungsfähigkeit des Modells.

WAS SIND PARAMETER

- Anzahl: LLMs enthalten oft Milliarden von Parametern, die die Komplexität und Kapazität des Modells bestimmen.
- Gewichtung: Parameter beeinflussen die Verarbeitung von Informationen im Modell.
- Anpassungsfähigkeit: Parameter werden während des Trainings optimiert, um Muster in den Daten zu erkennen.
- Speicherung von Wissen: Parameter speichern das erlernte Wissen des Modells.
- Einfluss auf die Leistung: Parameter beeinflussen direkt die Leistungsfähigkeit des Modells.

WELCHE ROLLE SPIELEN TOKENS

WELCHE ROLLE SPIELEN TOKENS

- Kleinste verarbeitbare Einheiten
- Sprache effizient analysieren und generieren
- Ganze Wörter, Wortteile oder einzelne Zeichen
- Durch Zerlegung sprachliche Muster erkennen und verstehen

WELCHE ROLLE SPIELEN TOKENS?

- OpenAI
- Claude
- DeepSeek

HUGGINGFACE

HUGGINGFACE 🤗

<https://huggingface.co/>

- Unternehmen mit Spezialisierung auf ML
- Baut Werkzeuge und Plattformen
- **Transformers Bibliothek**: Implementierungen von Transformer-Modellen für verschiedene Aufgaben
- Hugging Face Hub: Plattform zum Hosten von Modellen, Datensätzen und Anwendungen

HUGGING FACE HUB

Llama 3.2

- LICENSE.txt: Enthält die Lizenzbedingungen, unter denen das Modell veröffentlicht wird.
- README.md: Bietet eine Beschreibung des Modells, mit Details zu Training, Verwendung und Anwendungsbereichen.
- USE_POLICY.md: Nutzungsrichtlinien für das Modell, mit ethischen Überlegungen und Einschränkungen.
- config.json: Konfigurationsparameter des Modells, z.B. die Anzahl der Schichten oder die Größe der versteckten Zustände.
- generation_config.json: Standardparameter für die Textgenerierung, z.B. die maximale Länge oder die Sampling-Strategie.
- model-*.safetensors: Die trainierten Modellgewichte im safetensors-Format, (für sicheres und effizientes Speichern großer Tensoren)
- model.safetensors.index.json: Indexinformationen für die Modellgewichte in safetensors.
- special_tokens_map.json: Zuordnung spezieller Token wie [CLS], [SEP], [MASK] im Tokenizer.
- tokenizer.json: Vollständige Konfiguration des Tokenizers, einschließlich Vokabular und Mappings.
- tokenizer_config.json: Zusätzliche Konfigurationsparameter für den Tokenizer, z.B. Vor- und Nachverarbeitungsschritte.

WIE KANN ICH MIT EINEM LLM ARBEITEN?

AUSFÜHRUNG IM BROWSER MIT EINEM GEHOSTETEN MODELL

- OpenAI
- Ollama
- Vorteil: Schnell und einfach, hohe Modellqualität
- Nachteil: der API-Key liegt im Browser + Kosten + Datensicherheit

SERVERSEITIGE AUSFÜHRUNG MIT EINEM GEHOSTETEN MODELL

- OpenAI
- Ollama
- Vorteil: hohe Modellqualität, mehr Kontrolle
- Nachteil: Kosten, Datensicherheit, mehr Aufwand durch Server Implementierung

SERVERSEITIGE AUSFÜHRUNG MIT EIGENEM MODELL

- Ollama
- Vorteil: volle Kontrolle, keine Kosten, Datensicherheit
- Nachteil: Ressourcenbedarf, Implementierungsaufwand

WIE KANN ICH EIN LLM LOKAL AUSFÜHREN?

ANFORDERUNGEN

- CPU: für die Ausführung des Modells
- GPU: für die Ausführung des Modells
- RAM: vorhalten des Modells
- Größe: Llama 3.2 3B => Size 2GB => Memory 4GB

PLATTFORM FÜR DIE AUSFÜHRUNG

- GPT4All
- Ollama
- Llama.cpp
- LocalAI
- Jan

GPT4ALL

GPT4ALL

- Desktop-Applikation, um LLMs lokal auszuführen
- Keine API oder spezielle Hardware erforderlich
- Unterstützt viele Modelle
- Großer Wert auf Datenschutz

OLLAMA

INSTALLATION

- Auf allen gängigen Plattformen als Installer/Paket verfügbar
- Als Docker-Container verfügbar
- Erreichbar über Kommandozeile oder <http://localhost:11434>

OLLAMA COMMANDS

- serve: Start ollama
- pull: Pull a model from a registry
- run: Run a model, pull if necessary
- list: List models
- show: Show information for a model
- ps: List running models
- stop: Stop a running model
- rm: Remove a model
- create: Create a model from a Modelfile
- cp: Copy a model
- push: Push a model to a registry
- help: Help about any command

OPEN WEBUI

OPEN WEBUI

- Grafische Oberfläche für LLM-Runtimes
- Vergleichbar mit der ChatGPT UI
- Einfache Installation per Container
- Progressive Web App

```
docker run -d --network=host -v open-  
webui:/app/backend/data -e  
OLLAMA_BASE_URL=http://127.0.0.1:11434 --name  
open-webui --restart always ghcr.io/open-webui/open-  
webui:main
```

OLLAMA.JS

JAVASCRIPT-BIBLIOTHEK ZUR KOMMUNIKATION MIT OLLAMA

Installation: `npm install ollama`

```
import ollama from 'ollama';  
const response = await ollama.chat({  
  model: 'llama3.2',  
  messages: [{role: 'user', content: 'Was bedeutet LLM'}]  
})
```

MESSAGE TYPES

KONTEXT

KONTEXT

Menge an Text, die das Modell verarbeiten kann.
Beinhaltet die aktuelle Eingabe als auch vorherige
Informationen.

Self-Attention: Beziehungen zwischen Wörtern und
Sätzen des Kontexts analysieren, um Antworten zu
generieren.

Kontextgrößen:

- GPT 4o: 128k Tokens
- Llama 3.2: 128k Tokens
- Qwen 2.5-1M: 1M Tokens

KONTEXTGRÖSSEN

- Wieviel sind 128k bzw. 1M Tokens?
- Die **Normseite** hat 1500 Zeichen => ca. 250 Wörter
- Ein Token sind 0,75 Wörter, also hat eine Normseite ca. 333 Tokens.
- Eine Kontextgröße von 128k Tokens entspricht also ca. 384 Normseiten.
- Eine Kontextgröße von 1M Tokens entspricht ca. 3.000 Normseiten.
- Der Kontext ist zwar groß und wird auch immer größer, er ist aber nicht unendlich.

TOOLS

TOOLS

Problemstellung: Modelle haben keine aktuellen Informationen und können nicht auf geschützte Daten zugreifen

Lösungsmöglichkeit: Tools in Form von Funktionen implementieren, die die Modelle ausführen können.

Antwort fließt in den Kontext ein.

Einsatzgebiet: Aktuelle und sehr dynamische Informationen, Zugriff auf externe Datenquellen, Interaktion mit anderen Systemen.

CHATBOT

CHATBOT

- Chatbot: Dialogsystem, das natürliche Sprache versteht und generiert
- Einsatzgebiete: Kundenservice, Support, Information, Unterhaltung
- Vorteile: 24/7 Verfügbarkeit, Skalierbarkeit, Kosteneffizienz
- Nachteile: Limitierte Fähigkeiten, Datenschutz, Ethik
- Typische Client-/Server-Applikation

CHATBOT BACKEND

- Express.js
- Statische Auslieferung des Frontends
- API-Endpunkte für die Kommunikation mit dem Modell

CHATBOT FRONTEND

- HTML + JavaScript
- Element zur Anzeige der Nachrichten
- API-Endpunkte für die Kommunikation mit dem Modell

STREAMING

STREAMING

LLMs erzeugen Texte tokenweise. Bekannte Implementierungen wie ChatGPT zeigen die Antwort Wort für Wort an.

Ollama.js unterstützt Streaming, um die Antwort in Echtzeit anzuzeigen.

Mit HTTP können die Daten vom Server zum Client übertragen werden, bevor die Antwort vollständig ist.

VISION MODELL

VISION MODELL

- Viele Modelle sind auf Text spezialisiert
- Es gibt multi modale Modelle die neben Text auch Bilder unterstützen
- z.B. Llama 3.2 11B

OPENAI API

OPENAI API

- REST-API für den Zugriff auf verschiedene KI-Modelle
- Modelle: GPT, DALL-E, Whisper, Embedding-Modelle
- Unterstützt Streaming
- Authentifizierung erforderlich

MODELLOPTIMIERUNG

WARUM?

- Qualität der Antworten steigern
- Effizienz und Geschwindigkeit optimieren
- Domain-Spezifische Anpassungen
- Skalierbarkeit und Kostenkontrolle
- Sicherheit und Robustheit erhöhen
- Halluzinationen und Inkonsistenzen minimieren

WIE?

- Datengrundlage verbessern (Bessere Trainingsdaten, Fine-Tuning, Continual Learning)
- Modellarchitektur optimieren (Modellgröße und Parameter, Attention-Mechanismen verbessern, RAG)
- Effizienz und Geschwindigkeit steigern (Quantisierung und Pruning, Hardware, Knowledge Distillation)

EIN EIGENES MODELL TRAINIEREN

Das Training eines eigenen LLM bedeutet, ein neuronales Netz mit großen Mengen an Textdaten von Grund auf zu trainieren, um es für spezifische Aufgaben oder Domänen zu optimieren. Dies erfordert erhebliche Rechenressourcen, sorgfältige Datenaufbereitung und oft eine Anpassung der Modellarchitektur, um eine hohe Qualität und Effizienz zu gewährleisten.

WANN LOHNT SICH EIGENES TRAINING?

- Bestehende Modelle sind nicht gut genug (für Nischenanwendungen)
- Trainingsdaten sind zu sensibel
- Spezielle Architektur-Optimierungen notwendig

NACHTEILE

- Rechenintensiv: Benötigt spezialisierte Hardware (A100/H100 GPUs oder TPUs)
- Teuer: Trainingsressourcen kosten viel Geld
- Zeitaufwändig: Die Verarbeitung großer Datenmengen kostet viel Zeit

TRAINING

1. Datensammlung und -aufbereitung
2. Modellarchitektur wählen
3. Training

DATENSAMMLUNG UND -AUFBEREITUNG

Wikipedia, BooksCorpus, Common Crawl, ArXvid,
OpenWebText

- Tokenization (tiktoken, SentencePiece oder tokenizers)
- Duplikate entfernen und bereinigen
- Normalisierung

MODELLARCHITEKTUR WÄHLEN

Transformer-Architektur: nanoGPT, GPT-NeoX, RWKV
oder LLaMA-Architektur

- Small (125M Parameter): mit GPU trainieren
- Medium (1-3B Parameter): Mehrere GPUs
- Large (10B+ Parameter): Rechenzentrum oder TPU

TRAINING

- PyTorch
- TensorFlow
- JAX
- Hugging Face Transformers

FINE TUNING

FINE TUNING

- Anpassung eines vortrainierten Modells an spezifische Anforderungen oder Domänen
- Vorhandenes Wissen des Modells nutzen und darauf aufbauen
- Effizienter und ressourcenschonender

WANN LOHNT SICH FINE TUNING?

- Domänenspezifisches Wissen erforderlich
- Anpassung an Unternehmens- oder Produktdaten
- Stil- und Tonanpassung
- Verbesserung der Antwortqualität bei spezifischen Aufgaben
- Reduktion von Bias und Verbesserung der Ethik
- Performance-Optimierung für Edge- oder Offline-Einsätze

NACHTEILE VON FINE TUNING

- Hohe Kosten und Ressourcenbedarf
- Gefahr des „Catastrophic Forgetting“ - gelerntes Wissen geht verloren
- Risiko von Overfitting - zu kleiner Datensatz schränkt das Modell ein
- Komplexität bei der Datenaufbereitung

ABLAUF DES FINE TUNINGS

1. Auswahl des Basismodells
2. Aufbereitung und Vorbereitung der Trainingsdaten
3. Wahl der Fine-Tuning-Methode
4. Training des Modells mit neuen Daten
5. Evaluierung des fine-getunten Modells
6. Deployment des Fine-Tuned Modells

LORA

- Effizientes Fine Tuning von modellen
- Ursprüngliche Parameter werden eingefroren
- Einführung zusätzlicher Low-Rank-Matrizen, die trainiert werden
- Parameter des Modells und der Matrizen werden zusammengeführt

QUANTISIERUNG

QUANTISIERUNG

- Gewichte eines Modells werden normalerweise als 32-Bit-Float-Zahlen gespeichert
- Quantisierung: Umwandlung FP32 => INT8/INT4
- Geringerer Speicherverbrauch mit geringerer Genauigkeit
- Skalierungsfunktion um die Qualität zu verbessern
- QLoRA: Kombination aus Quantisierung und LoRA für noch effizienteres Fine Tuning

WAS BRINGT QUANTISIERUNG?

- Geringerer Speicherverbrauch: Llama 3.1 8B 16GB (FP16) - 4GB (INT4)
- Schnellere Berechnung
- Lauffähigkeit auf kleineren GPUs und CPUs

ARTEN DER QUANTISIERUNG

- Post-Training Quantization (PTQ) - schnell aber Genauigkeit geht verloren
- Quantization-Aware Training (QAT) - aufwändig aber höhere Genauigkeit
- Dynamic Quantization: Parameter werden vorab konvertiert, Aktivierungen werden zur Laufzeit dynamisch skaliert
- Static Quantization: Parameter werden während oder nach dem Training konvertiert

RAG

RAG

RETRIEVAL AUGMENTED GENERATION

Kombination von externem Wissens-Retrieval mit
generativer Textproduktion

RAG

- Retrieval: Suchmaschine oder Vektordatenbank sucht relevante Informationen aus einer Wissensquelle
- Augmentation: Gefundene Daten werden als Kontext an das LLM übergeben
- Generation: LLM nutzt den Kontext, um bessere Antworten zu generieren

VORTEILE VON RAG

- Aktuellere Antworten
- Reduzierte Halluzinationen
- Domänenspezifisches Wissen

OPTIMIERUNG

OPTIMIERUNG

- Will man es schneller machen? - Effizienz-Optimierung (Quantisierung, Pruning)
- Soll es genauer werden? - Bessere Trainingsdaten
- Braucht es Fachwissen? - Fine-Tuning oder RAG
- Muss es günstiger sein? - Skalierung & Quantisierung
- Soll es sicherer sein? - Bias-Reduktion & Prompt-Sicherheit

FRAGEN? ANMERKUNGEN? ANREGUNGEN?